MATH 231 – Linear Algebra I

The Computational Unit, Part 5 ▪ Matrix Multiplication as an Algorithm

Kiely Hall 326 ▪ Instructor: Kevin Bedoya

About this document. A short part. The matrix unit defined the matrix product and established its laws; none of that is repeated here. The question here is narrower and purely computational: taken as an *algorithm*, what does that definition actually instruct a machine to do, and how much work is it?

Part	Sections	Subject
1	§1–§4	operation counting, Big-O, floating point, error
2	§5–§10	k-nearest neighbours
3	§11–§15	k-means clustering
4	§16–§20	the Gram–Schmidt process
5	§21–§23	matrix multiplication as an algorithm

Assumed. From the matrix unit: what a matrix is and the index convention a_{ij} (§3.1); the definition of the matrix product and its dimension requirement (§8.1–§8.2). From this unit: operation counting and FLOPs (§1, §4.6) and Big-O (§2.7).

What you should be able to do afterwards. State the condition two matrices must satisfy before their product exists and give the shape of the result; write the pseudocode for the product and say what each of its three loops is responsible for; and count its operations and give its complexity.

21. The operation, reviewed

21.1. What is being computed

From the matrix unit. For $\mathbf{A} \in \mathbb{R}^{m\times p}$ and $\mathbf{B} \in \mathbb{R}^{p\times n}$, the product $\mathbf{C} = \mathbf{AB}$ is the $m \times n$ matrix with

$$c_{ij} \;=\; \sum_{k=1}^{p} a_{ik}\, b_{kj} \tag{1}$$

(§8.1 of the matrix unit). Entry c_{ij} is the dot product of row i of $\mathbf{A}$ with column j of $\mathbf{B}$.

Read (1) as an instruction rather than a description and it says: to fill in one entry of the answer, walk along a row of $\mathbf{A}$ and down a column of $\mathbf{B}$ in step, multiplying each pair and accumulating. Do that once for every position in the output grid.

That is already the algorithm. Everything below is bookkeeping around it.

21.2. What must be true first

Before any arithmetic happens, the shapes must be checked.

Precondition. **AB** exists only if the number of **columns of A** equals the number of **rows of B**.

$$(m \times \underbrace{p)\,(p}_{\text{must agree}} \times n) \longrightarrow m \times n$$

The reason is visible in (1): the sum pairs a_{ik} with b_{kj} for $k = 1, \ldots, p$, so the row of **A** and the column of **B** must have the same number of entries or some term has no partner. The shared value p is what the sum runs over.

An implementation that skips this check does not get a wrong answer — it reads memory that does not belong to it, and the failure is arbitrary. Verifying the shapes is the first line of the procedure for that reason, not out of tidiness.

Example 21.1.

$(4 \times 3)(3 \times 5)$	$\to$	4×5	defined
$(3 \times 5)(4 \times 3)$	$\to$	—	undefined: $5 \neq 4$
$(2 \times 2)(2 \times 2)$	$\to$	2×2	defined
$(1 \times n)(n \times 1)$	$\to$	1×1	defined — one dot product

The second line is the first with the operands swapped, and it does not exist. Order is part of the problem statement.

21.3. The size of the job

Three numbers describe the whole computation, and it is worth separating what each one controls:

Quantity	What it governs
m	the number of rows in the answer
n	the number of columns in the answer
p	the length of each dot product — the work *inside* one entry

So mn counts *how many* entries must be produced, and p counts *how expensive each one is*. Note that p does not appear in the output at all: it is summed away, and two matrices with a huge shared dimension can produce a tiny answer. A $(2 \times 10^6)(10^6 \times 2)$ product returns a 2×2 matrix after a great deal of work.

22. The algorithm

22.1. Pseudocode

Matrix multiplication

Input: $\mathbf{A} \in \mathbb{R}^{m \times p}$, $\mathbf{B} \in \mathbb{R}^{p \times n}$. **Output:** $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times n}$.

```
1: if cols(A) ≠ rows(B): stop — the product is undefined
2: for i = 1, ..., m:                          pick a row of the answer
3:     for j = 1, ..., n:                      pick a column of the answer
4:         s ← 0                               start the accumulator
5:         for k = 1, ..., p:                  walk the row and the column together
6:             s ← s + a_ik b_kj
7:         c_ij ← s                            one entry finished
8: return C
```

22.2. Reading the three loops

The structure is three nested loops, and each has a distinct job:

- The **outer two** (i and j) do no arithmetic whatsoever. They exist only to name a position in the output — together they visit each of the mn entries of $\mathbf{C}$ exactly once.
- The **inner one** (k) does all the work. For the position currently selected, it evaluates the dot product in (1).

So the algorithm is not really three loops deep in any conceptual sense. It is a double loop over the answer's entries, with a dot product performed at each stop. Line 6 is the only line that multiplies anything, and counting how many times it runs is the entire cost analysis of §23.

Remark (The accumulator). Lines 4–7 are the standard shape of a summation in code: set a running total to zero, add one term at a time, store the result when the loop ends. The variable s holds a partial sum that is meaningless until the loop finishes — only after k has run its full range does s equal c_{ij}.

A common confusion (Order of the loops). The three loops may be nested in any order without changing the answer, because addition is commutative and every c_{ij} accumulates the same p terms regardless of the sequence they arrive in. It does *not* follow that the orderings perform equally well on a real machine: they differ in the pattern by which they walk through memory, and that pattern can change the running time by a large factor. The operation count in §23 is identical for all of them, which is a reminder that an operation count is not a running time.

23. What it costs

23.1. One entry

Fix a position (i, j) and count what lines 4–7 do. The inner loop runs p times, and each pass performs one multiplication and one addition. The very first addition is to zero and could be skipped, so:

Operation	Count
multiplications $a_{ik}b_{kj}$	p
additions into s	$p-1$
per entry	$2p-1$ FLOPs

This is the dot-product count from §20.1, unchanged — one entry of a matrix product is one dot product of length p.

23.2. The total

There are mn entries and each costs $2p-1$, and no entry's work is shared with any other's. So

$$\text{total} \;=\; mn\,(2p-1) \;=\; 2mnp - mn \;\approx\; 2mnp \text{ FLOPs.}$$

Theorem 23.1 (Cost of matrix multiplication)**.** *Multiplying $\mathbf{A} \in \mathbb{R}^{m\times p}$ by $\mathbf{B} \in \mathbb{R}^{p\times n}$ by the algorithm of §22.1 takes $\approx 2mnp$ FLOPs, so it runs in $O(mnp)$ time.*

Every one of the three dimensions appears exactly once, and each for a reason already identified in §21.3: m and n because that many entries must be filled, p because that is the length of each dot product.

23.3. The square case

When all three dimensions are equal — two $n \times n$ matrices, the case that comes up most — set $m = n = p$:

$$2mnp \;=\; 2n^3, \qquad \text{so the cost is } O(n^3).$$

The cubic is easy to see directly from the pseudocode: three nested loops, each running n times, with constant work at the innermost level.

Remark. Doubling n multiplies the work by roughly eight. Two 1000×1000 matrices — a million entries each, which is not a large problem — already require about 2×10^9 floating-point operations.

Remark (The same $2n^3$ as Gram–Schmidt). Part 4 found that orthonormalising n vectors in $\mathbb{R}^n$ also costs about $2n^3$ FLOPs. The agreement is not a coincidence in any deep sense, but it is not an accident either: both algorithms spend their time computing dot products of length n, and both perform on the order of n^2 of them. The structures differ — matrix multiplication does exactly n^2 dot products, while Gram–Schmidt does about $n^2/2$ and pays for the rest in subtractions — yet they land on the same leading term. Where an algorithm's cost comes from is often more informative than the count itself.

23.4. Is cubic the best possible?

A natural assumption is that n^3 is forced — the answer has n^2 entries and each looks like it needs n operations, so what could be saved?

It is not forced.

Looking ahead. There are algorithms that multiply $n \times n$ matrices in asymptotically fewer than n^3 operations. The first, due to Strassen, does it in about $n^{2.81}$ by computing a 2×2 product with seven multiplications instead of the obvious eight and applying that trick recursively. Later

methods have pushed the exponent lower still, though the practical ones remain close to Strassen. The exponent that is actually optimal is not known — it cannot be below 2, since the output alone has n^2 entries that must each be written, and closing the gap between 2 and the best known value is a genuine open problem.

For this course, $O(n^3)$ is the operative figure: it is what the straightforward algorithm costs, it is what the standard library routines effectively deliver at the sizes we will meet, and it is the number to reach for when estimating whether a computation is affordable.